파머완 1장

01 머신러닝의 개념

머신러닝이란?

데이터를 기반으로 패턴을 학습하고 결과를 예측하는 알고리즘 기법

머신러닝 알고리즘

데이터를 기반으로 통계적인 신뢰도 강화하고 수학적 기법을 적용해 데이터 내의 패턴을 스스로 인지하고 신뢰도 있는 예측 결과를 도출함.

머신러닝의 분류

지도학습(Supervised Learning)

- 분류(Classification)
- 회귀(Regression)
- 추천 시스템
- 시각/음성 감지/인지
- 텍스트 분석, NLP

비지도학습(Un-supervised Learning)

- 클러스터링
- 차원 축소
- 강화학습

데이터 전쟁

머신러닝 단점

데이터에 매우 의존적. Garbage In, Garbage out.

데이터를 효율적으로 가공, 처리, 추출해 최적의 데이터 준비 능력 중요.

파이썬과 R 기반의 머신러닝 비교

파이썬	***R***
개발 전문 프로그램 언어	통계 전용 프로그램 언어
다양한 분야	통계 분석 특화

현재는 파이썬이 더 트렌드!

02 파이썬 머신러닝 생태계를 구성하는 주요 패키지

- 머신러닝 패키지
  - 대표 패키지: 사이킷런(Scikit-Learn)
- 행렬/선형대수/통계 패키지
  - 이론적 백그라운드
  - 대표 패키지: NumPy (행렬, 선형대수)
  - 대표 패키지: SciPy (자연과학, 통계)
- 데이터 핸들링
  - 대표 패키지: 판다스
- 시각화
  - 대표 패키지: 맷플롯립(Matplotlib)
  - 효율성 보완된 패키지: 시본(Seaborn) ← 맷플롯립 기본지식 필요
- 아이파이썬 툴
  - 대화형 파이썬 툴
  - 대표 툴: 주피터 노트북(Jupyter Notebook)

03 넘파이

넘파이(Numerical Python)

- 파이썬에서 선형대수 기반의 프로그램 제작 지원 패키지
- 대량 데이터의 배열 연산 가능하게 함 → 빠른 배열 연산 속도
- 저수준 언어 기반 호환 API 제공

- 파이썬 자체의 수행 성능 제약 → C/C++기반의 코드를 넘파이에서 호출 (통합)
- 판다스의 편리성에는 미치지 못함

넘파이 ndarray 개요

ndarray

- 넘파이의 기반 데이터 타입
- 넘파이에서 다차원(Multi-dimention) 배열 생성 및 연산 수행을 가능하게 함.

array() 함수

- 인자를 입력받아 ndarray로 변환
- 생성된 ndarray 배열의 shape 변수는 ndarray의 크기를 튜플 형태로 보유 → ndarray 배열의 차원까지 알 수 있음

np.array() 사용법

- ndarray로 변환할 객체 인자로 입력 → ndarray 반환
- ndarray.shape : 차원과 크기를 튜플 형태로 나타냄

array	***shape (로우,칼럼)***	***차원***	***데이터 수***
[1, 2, 3]	(3,)	1차원 array	3
[[1, 2, 3]. [2, 3, 4]]	(2,3)	2차원 array	6
[[1,2,3]]	(1,3)	2차원 array	3

ndarray의 데이터 타입

- 숫자 값, 문자열 값, 불 값 등이 모두 가능
- 숫자형 : int형(8bit, 16bit, 32bit) / unsigned int형(8bit, 16bit, 32bit), float형(16bit, 32bit, 64bit, 128bit)
- ndarray내에는 같은 데이터 타입만 가능
  - 다른 데이터 유형 섞여 있는 리스트 → 더 큰 데이터 타입으로 일괄 형 변환
- dytpe 속성으로 데이터 타입 확인
- astype() 메서드로 데이터값 타입 변경

ndarray를 편리하게 생성하기 - arange, zeros, ones

특정 크기와 차원을 가진 ndarray를 연속값이나 0또는 1로 초기화해 생성 필요

→ arange(), zeros(), ones()

arange()

- range()와 유사한 기능
- array를 range()로 표현
- 0부터 함수 인자 값 -1까지의 값을 순차적으로 ndarray의 데이터값으로 변환

zeros()

- 함수 인자로 튜플 형태의 shape 값을 입력
  → 모든 값을 0으로 채운 해당 shape를 가진 ndarray 반환

ones()

- 함수 인자로 튜플 형태의 shape 값을 입력
  → 모든 값을 1로 채운 해당 shape를 가진 ndarray 반환

reshape() 메서드

- ndarray를 특정 차원 및 크기로 변환
- 변환을 원하는 크기를 함수 인자로 부여

넘파이의 ndarray의 데이터 세트 선택하기 - 인덱싱 (Indexing)

1. 특정한 데이터만 추출: 원하는 위치의 인덱스 값 지정 시 해당 위치의 데이터 반환
2. 슬라이싱(Slicing): 슬라이싱은 연속된 인덱스상의 ndarray를 추출하는 방식
   ':' 기호 사이에 시작 인덱스&종료 인덱스 표시 시,
   시작 인덱스에서 종료 인덱스-1 위치 데이터의 ndarray 반환
3. 팬시 인덱싱(Fancy Indexing): 일정한 인덱싱 집합을 리스트 또는 ndarray 형태로 지정해 해당 위치 데이터의 ndarray 반환
4. 불린 인덱싱(Boolean Indexing): True/False 값 인덱싱 집합을 기반으로 True에 해당하는 인덱스 위치 데이터의 ndarray 반환

단일 값 추출

- ndarray 객체에 해당하는 위치의 인덱스 값을 [ ] 안에 입력

- 인덱스에 마이너스 기호를 이용하면 맨 뒤에서부터 데이터를 추출

  → 인덱스 -1 : 맨 뒤의 데이터 값

슬라이싱

- ':' 기호를 이용해 연속한 데이터를 슬라이싱해서 추출
- 슬라이싱, 팬시/불린 인덱싱으로 추출된 데이터 세트는 ndarray 타입 (단일 값 추출 제외)
- ':' 사이의 시작, 종료 인덱스 생략 가능
  1. ' : ' 앞에 시작 인덱스 생략 시 맨 처음 인덱스인 0으로 간주
  2. ':' 뒤에 종료 인덱스 생략 시 맨 마지막 인덱스로 간주
  3. ' : ' 앞/뒤에 시작/종료 인덱스 생략 시 맨 처음/맨 마지막 인덱스로 간주

펜시 인덱싱

- 리스트나 ndarray로 인덱스 집합 지정 시 해당 위치 인덱스에 해당하는 ndarray를 반환

불린 인덱싱

- 조건 필터링과 검색 동시 가능
- ndarray의 인덱스를 지정하는 [ ] 내에 조건문을 그대로 기재
- 동작하는 단계
  1. ： array1d 〉5와 같이 ndarray의 필터링 조건을 [ ] 안에 기재
  2. False 값은 무시하고 True 값에 해당하는 인덱스값만 저장

     (True값 자체인 1을 저장하는 것이 아니라 True값을 가진 인덱스를 저장)
  3. 저장된 인덱스 세트로 ndarray 조회

행렬의 정렬 - sort()와 argsort()

sort()

- np.sort()
  - 넘파이에서 sort() 호출
  - 원 행렬 유지한 채 정렬된 행렬 반환

- ndarray.sort()
    - 행렬 자체에서 sort() 호출
    - 원 행렬 자체를 정렬한 형태로 변환
    - 반환값: 0

정렬된 행렬의 인덱스 반환

- 원본 행렬 정렬 후 기존 원본 행렬의 원소에 대한 인덱스 필요 시 np.argsort() 이용
- np.argsort()는 정렬 행렬의 원본 행렬 인덱스를 ndarray 형으로 반환

argsort()

- 넘파이에서 활용도 매우 높음
- 넘파이의 ndarray는 실제 값과 그 값이 뜻하는 메타 데이터를 별도의 ndarray로 각각 가져야함

선형대수 연산 - 행렬 내적과 전치 행렬 구하기

행렬 내적

- 행렬 곱
- 두 행렬 사이의 내적은 np.dot()을 이용해 계산
- 두 행렬의 내적: 왼쪽 행렬의 로우와 오른쪽 행렬의 캄럼의 원소를 순차로 곱한 후 더한 값

전치 행렬

- 원 행렬에서 행과 열 위치를 교환한 원소로 구성한 행렬

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$$

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$$

*행렬A와 행렬A의 전치행렬

데이터 핸들링 - 판다스

판다스(Padas)

- 파이썬에서 데이터 처리를 위해 존재하는 가장 인기 있는 라이브러리
- 일반적으로 대부분의 데이터 세트는 2차원 데이터
- 2차원 데이터를 효율적으로 가공/처리할 수 있는 다양하고 훌륭한 기능을 제공
- 많은 부분이 넘파이 기반으로 작성됐으나 훨씬 유연하고 편리한 데이터 핸들링 가능

DataFrame

- 데이터뿐만 아니라 칼럼의 타입, Null 데이터 개수, 데이터 분포도 등의 메타 데이터도 조회 가능
- info() 메서드
  - 총 데이터 건수, 데이터 타입, Null 건수를 알 수 있음
- describe() 메서드
  - 칼럼별 숫자형 데이터값의 n-percentile 분포도, 평균값, 최댓값, 최솟값을 나타냄
  - 숫자형(int, float 등) 칼럼의 분포도만 조사
  - object 타입의 칼럼은 자동으로 제외
- value_coumts() 메서드
  - 해당 칼럼값의 유형과 건수를 확인

- 지정된 칼럼의 데이터값 건수 반환

DataFrame과 리스트, 딕셔너리, 넘파이 ndarray 상호 변환

DataFrame ↔ 파이썬 리스트 / 딕셔너리, 넘파이 ndarray (상호 변환 가능, 2차원 이하의 데이터만)

DataFrame 데이터 삭제

- drop() 메서드 이용
- axis 값에 따라 특정 칼럼 또는 행 드롭
  - axis 0 : 로우 방향 축
  - axis 1: 칼럼 방향 축
- inplace
  - False(Default): 자기 자신의 데이터는 삭제하지 않음
  - True: 자신의 데이터 삭제

Index 객체

- DataFrame, Series의 레코드를 고유하게 식별하는 객체
- DataFrame, Series에서 Index 객체만 추출 시 DataFrame.index 또는 Series.index 속성 이용

reset_index()

- 연속된 int 숫자형 데이터가 아닌 인덱스를 연속 int 숫자형 데이터로 만들 때 주로 사용
- Series에 reset_index()를 적용하면 Series가 아닌 DataFrame이 반환

[ ]연산자

넘파이

- 행의 위치, 열의 위치, 슬라이싱 범위 등을 지정해 데이터 가져옴

DataFrame

- DataFrame 뒤 '[ ]' 안에는 칼럼명 문자/리스트 객체나 인덱스로 변환 가능한 표현식만 가능
- DataFrame 바로 뒤 [ ] 연산자는 넘파이의 [ ]나 Series의 [ ]와 다름

- DataFrame 바로 뒤 [ ] 내 입력값은 칼럼명/리스트를 지정해 칼럼 지정 연산에 사용하거나 불린 인덱스 용도로만 사용해야 함
- DataFrame[0:2]와 같은 슬라이싱 연산으로 데이터 추출은 하지 않는 게 좋음

iloc[ ]

- 위치(Location) 기반 인덱싱 방식
  - 행과 열 위치를, 0을 출발점으로 하는 세로축, 가로축 좌표 정숫값으로 지정하는 방식
- 행/열의 좌표 위치에 해당 값으로 정숫값 또는 정수형의 슬라이싱, 팬시 리스트 값을 입력 필요
- 열 위치에 -1을 입력하여 DataFrame의 가장 마지막 열 데이터를 가져오는 데 자주 사용

loc[ ]

- 명칭(Label) 기반 인덱싱 방식
  - 데이터 프레임의 인덱스 값으로 행 위치를, 칼럼의 명칭으로 열 위치를 지정하는 방식
- 사용 시 행 위치에 DataFrame의 인덱스 값 들어감
- 슬라이싱 기호를 적용하면 종료 값-1이 아니라 종료 값까지 포함

불린 인덱싱

- 매우 편리한 데이터 필터링 방식
- 처음부터 가져올 값을 조건으로 [ ] 내에 입력하면 자동으로 원하는 값을 필터링
- [ ], loc[ ]에서 공통으로 지원

정렬

DataFrame / Series 정렬

→ sort_values() 메서드 이용

sort_valuse()

- 주요 입력 파라미터: by, ascending, inplace
- by로 특정 칼럼을 입력하면 해당 칼럼으로 정렬을 수행

Aggregation 함수 적용

- DataFrame에서 바로 aggregation을 호출할 경우 모든 칼럼에 해당 aggregation을 적용
- 특정 칼럼에 aggregation 함수 적용 시 DataFrame에 대상 칼럼들만 추출해 적용

groupby() 적용

- SQL과 판다스 모두 분석 작업에 많이 활용
- DataFrame의 groupby() 사용 시, 입력 파라미터 by에 칼럼을 입력하면 대상 칼럼으로 groupby됨
- DataFrame에 groupby()를 호출 시 DataFrameGroupBy라는 형태의 DataFrame 반환